CVE-2026-31431 – REMEDIATION COMPARISON – 2026-04-30

Who Can Fix Copy Fail – and How Fast

Kernel patch status, pre-patch mitigations, and fleet deployment across every major Linux distribution

---

THE BOTTOM LINE

As of April 30, 2026 – one day after disclosure – **no enterprise distribution has shipped a kernel patch.** Every production RHEL, Ubuntu, SUSE, and Amazon Linux system is unpatched. The differentiator is not who patches first. It is: **what can you do right now before the patch ships, and how fast can you deploy it when it does?**

Kernel Patch Status (as of 2026-04-30)

Distribution	Patch Status	Fixed Kernel	Source
Red Hat Enterprise Linux 10	Pending	–	Bugzilla 2460538 (Status: NEW)
Red Hat Enterprise Linux 9	Pending	–	Bugzilla 2460538
Red Hat Enterprise Linux 8	Pending	–	Bugzilla 2460538
Ubuntu 24.04 / 22.04 / 20.04	Pending	–	USN tracker
SUSE 15 SP7 / 16.0 / 16.1	Pending	–	SUSE CVE page (CVSS 7.8)
Amazon Linux 2 / 2023	Pending Fix	–	ALAS tracker
Debian Bullseye (11)	Vulnerable	–	DSA tracker
Debian Bookworm (12)	Vulnerable	–	DSA tracker
Debian Trixie (13)	Fixed	6.12.85-1	DSA-6238-1
Fedora (current)	Fixed	6.19.12+	Upstream merge
Android	No bulletin yet	–	Monthly security bulletin process

SUSE policy note: SUSE has declared that they “will no longer fix all CVEs in the Linux Kernel” (TID 21496). While this CVE is marked Important/7.8, SUSE’s selective patching policy introduces uncertainty about timeline.

Pre-Patch Mitigations: Who Has What

Mitigation	RHEL	Ubuntu	SUSE	Amazon Linux
Module blacklist (`algif_aead`)	No — compiled in (`=y`)	Depends on build	Depends on build	Depends on build
Custom MAC policy for `AF_ALG` sockets	Yes — SELinux custom module	Per-profile only — `deny network alg,` in each AppArmor profile	Per-profile only — `deny network alg,` in each AppArmor profile	SELinux available but not default
Container seccomp for `AF_ALG`	Yes — Podman custom seccomp	Docker custom seccomp (manual)	Docker custom seccomp (manual)	ECS/EKS seccomp (complex)
systemd `RestrictAddressFamilies`	Yes — per-service override	Yes — per-service override	Yes — per-service override	Yes — per-service override
Fleet deployment of mitigation	Ansible + Satellite — minutes	Manual or ad-hoc tooling	SUSE Manager (if deployed)	SSM (AWS-managed only)

RHEL Remediation: Three Layers, All Available Today

1 Containers: Block `AF_ALG` at the seccomp layer (5 minutes)

Create a custom seccomp profile that blocks `AF_ALG` socket creation. Apply to all Podman containers.

```
{
  "defaultAction": "SCMP_ACT_ALLOW",
  "syscalls": [
    {
      "names": ["socket"],
      "action": "SCMP_ACT_ERRNO",
      "errnoRet": 93,
      "args": [
        {
          "index": 0,
          "value": 38,
          "op": "SCMP_CMP_EQ"
        }
      ]
    }
  ]
}
```

```
# Save as /etc/containers/seccomp-no-af-alg.json
# Apply to containers:
podman run --security-opt seccomp=/etc/containers/seccomp-no-af-alg.json ...

# Or set as default in /etc/containers/containers.conf:
# [containers]
# seccomp_profile = "/etc/containers/seccomp-no-af-alg.json"
```

2 Services: `RestrictAddressFamilies` via systemd (5 minutes per service)

For any systemd-managed service, restrict socket creation to only the families it needs.

```
# /etc/systemd/system/myservice.service.d/no-af-alg.conf
[Service]
RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6 AF_NETLINK
```

```
# Deploy fleet-wide via Ansible:
- name: Block AF_ALG for critical services
  ansible.builtin.copy:
    dest: "/etc/systemd/system/{{ item }}.service.d/cve-2026-31431.conf"
    content: |
      [Service]
      RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6 AF_NETLINK
  loop:
    - httpd
    - postgresql
    - nginx
  notify: reload systemd

- name: Reload systemd
  ansible.builtin.systemd:
    daemon_reload: yes
  listen: reload systemd
```

3 Fleet Deployment: Ansible Automation Platform + Red Hat Satellite

This is where RHEL pulls ahead of every other distribution. The mitigation above — seccomp profiles and systemd overrides — can be pushed to **every managed host in minutes** using infrastructure that RHEL customers already have deployed.

```
# Full CVE-2026-31431 mitigation playbook
# Push via Ansible Automation Platform or run ad-hoc against Satellite inventory

- name: CVE-2026-31431 pre-patch mitigation
  hosts: all
  become: true
  tasks:

    - name: Deploy seccomp profile blocking AF_ALG
      ansible.builtin.copy:
        dest: /etc/containers/seccomp-no-af-alg.json
        mode: '0644'
        content: |
          {
            "defaultAction": "SCMP_ACT_ALLOW",
            "syscalls": [
              {
                "names": ["socket"],
                "action": "SCMP_ACT_ERRNO",
                "errnoRet": 93,
                "args": [
                  { "index": 0, "value": 38, "op": "SCMP_CMP_EQ" }
                ]
              }
            ]
          }
      when: ansible_facts.packages['podman'] is defined

    - name: Set seccomp default in containers.conf
      community.general.ini_file:
        path: /etc/containers/containers.conf
        section: containers
        option: seccomp_profile
        value: '"/etc/containers/seccomp-no-af-alg.json"'
      when: ansible_facts.packages['podman'] is defined

    - name: Deploy systemd overrides for exposed services
      ansible.builtin.copy:
        dest: "/etc/systemd/system/{{ item }}.service.d/cve-2026-31431.conf"
        mode: '0644'
        content: |
          [Service]
          RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6 AF_NETLINK
      loop: "{{ cve_31431_services | default(['httpd', 'nginx', 'postgresql', 'mariadb', 'tomcat'])
```

```
    when: ansible_facts.services[item + '.service'] is defined
    notify: reload systemd

  - name: Reload systemd
    ansible.builtin.systemd:
      daemon_reload: yes
    listen: reload systemd
```

DEPLOYMENT COMPARISON

RHEL + Ansible + Satellite: Write the playbook once. Push to thousands of hosts via Ansible Automation Platform job template. Satellite inventory ensures you hit every registered system. Total time: minutes.

Ubuntu: No equivalent fleet orchestration included. Landscape exists but is separately licensed and less common. Most shops use ad-hoc SSH loops or Ansible without a centralized inventory.

SUSE: SUSE Manager can push Salt states, but adoption is lower and SUSE's selective CVE patching policy means the kernel fix itself may never ship.

Amazon Linux: AWS Systems Manager (SSM) works for EC2/ECS, but only for AWS-managed instances. Hybrid and on-prem Amazon Linux systems require separate tooling.

When the Kernel Patch Ships

Pre-patch mitigations buy time. The real question is: **when the vendor ships the fixed kernel, how fast can you deploy it across your entire fleet?**

RHEL: Satellite Content View Workflow

1. **Errata arrives** – Red Hat publishes the kernel advisory (RHSA) with the fix
2. **Satellite syncs** – Your Satellite server automatically pulls the new packages
3. **Content view promotion** – Stage → Test → Production lifecycle environments
4. **Ansible job template** – Push `dnf update kernel` to all hosts matching the CVE filter

5. **Reboot orchestration** — Rolling reboots via Ansible with health checks between batches

```
# Satellite: filter errata by CVE and promote
hammer content-view filter create \
  --name "CVE-2026-31431" \
  --type erratum \
  --content-view "RHEL-Production" \
  --organization "MyOrg"

hammer content-view filter rule create \
  --content-view-filter "CVE-2026-31431" \
  --content-view "RHEL-Production" \
  --errata-id "RHSA-2026:XXXX" \
  --organization "MyOrg"

hammer content-view publish \
  --name "RHEL-Production" \
  --organization "MyOrg"

hammer content-view version promote \
  --content-view "RHEL-Production" \
  --to-lifecycle-environment "Production" \
  --organization "MyOrg"
```

RHEL Image Mode: Atomic Update with Rollback

For RHEL systems deployed via **bootc image mode**, the kernel update is an atomic image swap with automatic rollback if the new image fails to boot.

```
# Build updated image with patched kernel
podman build -t registry.internal/rhel10-patched:cve-31431 .

# Push to registry
podman push registry.internal/rhel10-patched:cve-31431

# On each host (or via Ansible):
bootc upgrade
systemctl reboot

# If boot fails, GRUB automatically rolls back to previous image
```

WHY THIS MATTERS

On Ubuntu or SUSE, a failed kernel update means manual GRUB recovery or booting from rescue media. On RHEL image mode, it's automatic. At fleet scale, the difference between "one bad kernel bricks 200 servers" and "200 servers automatically roll back and page the admin" is measured in downtime hours and incident severity.

Post-Patch Deployment Speed Comparison

Capability	RHEL	Ubuntu	SUSE	Amazon Linux
Centralized patch management	Satellite (included)	Landscape (separate license)	SUSE Manager	SSM (AWS only)
Errata-level targeting	Yes — filter by CVE ID	No — package-level only	Yes (if SUSE patches it)	ALAS advisory
Lifecycle environments	Stage → Test → Prod	Manual repo management	Channel-based	N/A
Orchestrated fleet reboot	Ansible rolling reboot	Manual or ad-hoc	Salt-based	SSM maintenance windows
Atomic rollback on failed boot	bootc image mode	No	Btrfs snapshots (manual)	No
CVE assessment automation	Red Hat Insights + Lightspeed	No equivalent	No equivalent	Inspector (limited)

Red Hat Insights: Know Before You Patch

Red Hat Insights — included with every RHEL subscription — provides CVE assessment across your entire fleet before the patch ships. For CVE-2026-31431:

- **Vulnerability dashboard:** Shows which systems are affected based on kernel version and config
- **Advisor recommendations:** Pre-built remediation playbooks downloadable directly into Ansible Automation Platform
- **Compliance reporting:** Demonstrate to auditors which systems have mitigations applied and which are pending kernel update

No other distribution provides a comparable integrated vulnerability assessment and remediation pipeline at zero additional cost.

Summary: The RHEL Advantage

THREE THINGS RHEL CAN DO TODAY THAT NO ONE ELSE CAN MATCH

1. **SELinux custom module** — Block `AF_ALG` socket creation system-wide with a single policy module. AppArmor (Ubuntu, SUSE) can deny `AF_ALG` per-profile (`deny network alg,`) but requires updating every confined application's profile individually — no system-wide equivalent.
2. **Ansible + Satellite fleet deployment** — Push the seccomp and systemd mitigations to every managed host in minutes, with inventory-driven targeting and compliance reporting. No other distro bundles equivalent orchestration.
3. **bootc atomic rollback** — When the kernel patch ships, deploy it as an atomic image update. Failed boot = automatic rollback. At scale, this eliminates the "bad kernel, 200 servers down" scenario entirely.

The kernel patch race is a wash — every enterprise vendor is pending. What separates RHEL is the operational machinery around the patch: **the ability to mitigate now, deploy safely when the fix ships, and prove compliance throughout.**

Jim O'Donnell • Red Hat • RHEL Specialist Solutions Architect • 2026-04-30